LAPRAK TEKNOLOGI IOT II

PRATIKUM 4 & 5

Oleh:

NAMA : TAUFIK FADILLA

NPM : 25782114

KELAS : TRI – 3D

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, S.Kom

TEKNOLOGI REKAYASA INTERNET

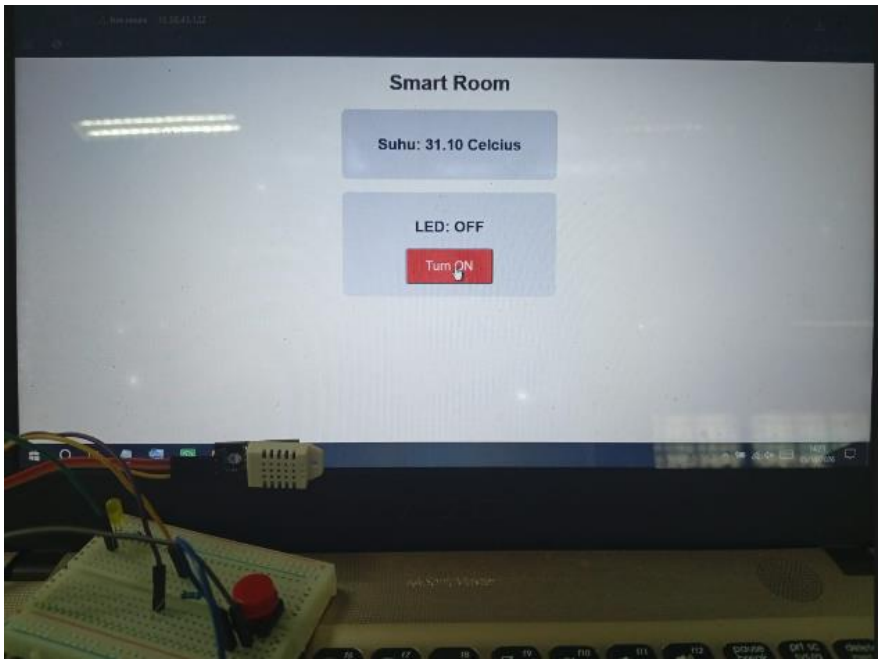
JURUSAN TEKNOLOGI INFORMASI

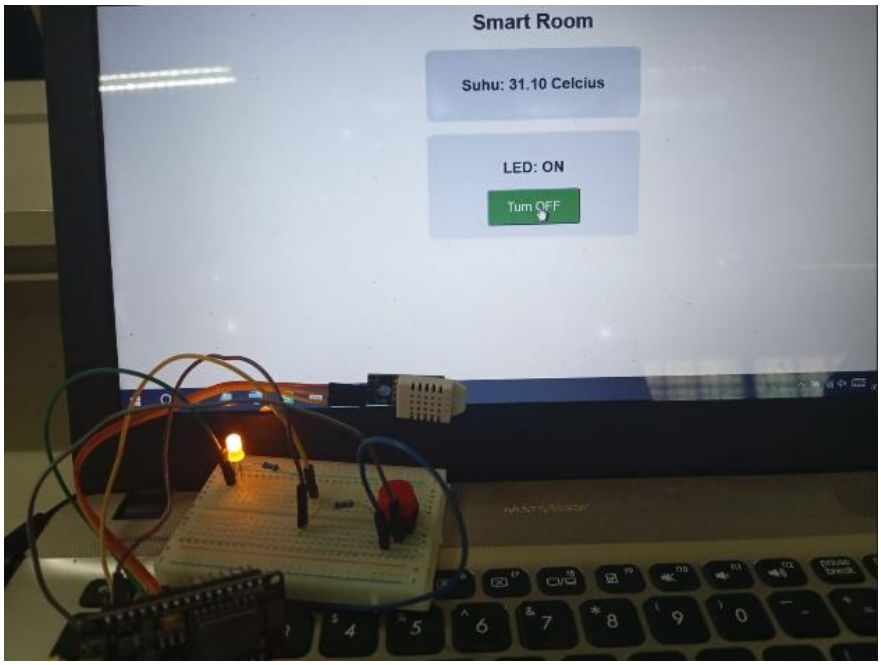
POLITEKNIK NEGERI LAMPUNG

2025/2026

4.5.1 Praktikum 1: *Real-Time Dashboard* & Sinkronisasi Tombol Fisik

Gambar





Hasil

Sistem berhasil menghubungkan NodeMCU ke jaringan Wi-Fi lokal dalam mode *Station* (STA) dan membuka layanan *Web Server* pada port 80. Saat alamat IP NodeMCU diakses melalui *browser*, sistem menampilkan antarmuka HTML bertuliskan "Smart Room" yang memuat pembacaan angka suhu secara *real-time* dari sensor DHT (tercatat 31.10 Celcius) tanpa perlu memuat ulang keseluruhan halaman. Berdasarkan dokumentasi foto, sistem sinkronisasi dua arah berhasil dieksekusi: saat tombol kendali berlabel "Turn ON" atau "Turn OFF" ditekan melalui antarmuka *browser*, status teks berubah seketika mengikuti warna kelas CSS-nya, dan mikrokontroler

mengeksekusi logika pin untuk menghidupkan dan mematikan komponen LED fisik di *breadboard* secara asinkron.

4.6 Latihan

Latihan 1 (Teori Jaringan): Teknologi WebSockets jauh lebih hemat konsumsi data dan mencegah layar berkedip (flickering) karena menggunakan protokol komunikasi yang membuka koneksi persisten dan dua arah (full-duplex) secara konstan antara browser (Klien) dan NodeMCU (Server). Berbeda dengan arsitektur HTTP standar dan Meta Refresh yang memaksa browser memuat ulang seluruh kerangka web secara keseluruhan, WebSockets memungkinkan NodeMCU untuk secara proaktif mengirimkan paket data kecil berformat JSON hanya ketika terjadi perubahan. Melalui manipulasi JavaScript, browser hanya akan memperbarui teks angka atau status yang diperlukan secara spesifik tanpa memuat ulang elemen halaman web lainnya.

Latihan 2 (Analisis Waktu/Timer): Memanggil sintaks delay(3000) di dalam siklus void loop() akan membuat CPU mikrokontroler membeku (tertidur) selama 3 detik, yang secara sistematis memblokir respons WebSocket. Akibat dari pemblokiran CPU (blocking) ini, NodeMCU kehilangan kemampuannya untuk bersiaga, sehingga tidak dapat merespons sentuhan tombol mekanik fisik maupun mengeksekusi instruksi dari antarmuka jaringan internet secara instan (real-time).

Latihan 3 (Modifikasi Front-End):

: Target elemen ini digunakan untuk mengubah teks indikator status perangkat keras menjadi "ON" atau "OFF".

<button id="toggleBtn">: Target elemen ini digunakan untuk mengubah teks pada tombol menjadi "Turn OFF" atau "Turn ON", serta mengubah kelas warnanya.

Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan):

1. **Sisi Back-End (C++)**

```
15
16  // Variabel Pelacak Status (State & Cache)
17  bool ledState = false;
18  String currentTemp = "--";
19  String currentHum = "--";
20  int buttonState;
21  int lastButtonState = LOW;
22  unsigned long lastDebounceTime = 0;
23  const unsigned long debounceDelay = 50;
24  unsigned long lastTime = 0;
```

```
115 void notifyClients() {
116   String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", ";
117   jsonString += "\"suhu\":\"" + currentTemp + "\", ";
118   jsonString += "\"hum\":\"" + currentHum + "\"}"; // Menyisipkan data kelembapan
119   ws.textAll(jsonString);
120 }
```

```
  // 3. Baca DHT setiap 3 detik di memori cache, tanpa memblokir CPU
  if ((millis() - lastTime) > 3000) {
    float t = dht.readTemperature();
    float h = dht.readHumidity(); // Membaca nilai kelembapan

    if(!isnan(t) && !isnan(h)) {
      currentTemp = String(t);
      currentHum = String(h); // Menyimpan nilai kelembapan ke variabel
      notifyClients(); // Push pembaruan otomatis ke browser
    }
    lastTime = millis();
  }
}
```

2. **Sisi Front-End (HTML & JavaScript)**

```
<body>
  <h1>Smart Room</h1>
  <div class="card">
    <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
  </div>
  <!-- Tambahan kartu penampil kelembapan -->
  <div class="card">
    <h2>Kelembapan: <span id="humValue">--</span> %</h2>
  </div>

```

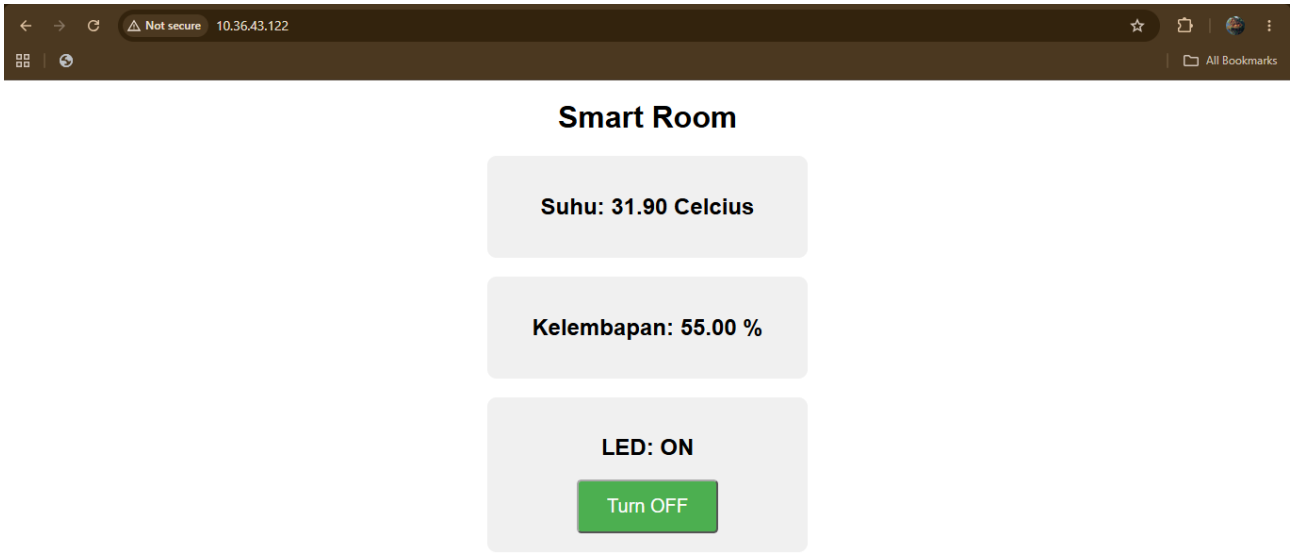
```
// Menangkap paket data JSON yang "didorong" (PUSH) oleh C++ NodeMCU
function onMessage(event) {
  var dataObj = JSON.parse(event.data);

  // Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML
  if(dataObj.suhu !== undefined) {
    document.getElementById('tempValue').innerHTML = dataObj.suhu;
  }

  // Menyuntikkan teks angka Kelembapan ke dalam id "humValue" HTML
  if(dataObj.hum !== undefined) {
    document.getElementById('humValue').innerHTML = dataObj.hum;
  }

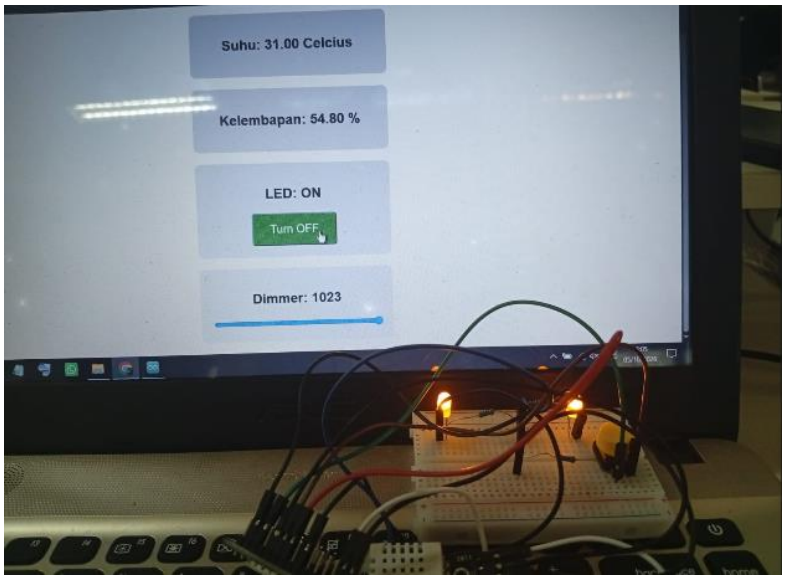
  // Merombak UI Tombol & Teks sesuai status hardware
  if(dataObj.led !== undefined) {
    var btn = document.getElementById('toggleBtn');
    var status = document.getElementById('ledStatus');
    if(dataObj.led == "1"){
      status.innerHTML = "ON";
      btn.innerHTML = "Turn OFF";
      btn.className = "btn-on";
    } else {
      status.innerHTML = "OFF";
      btn.innerHTML = "Turn ON";
      btn.className = "btn-off";
    }
  }
```

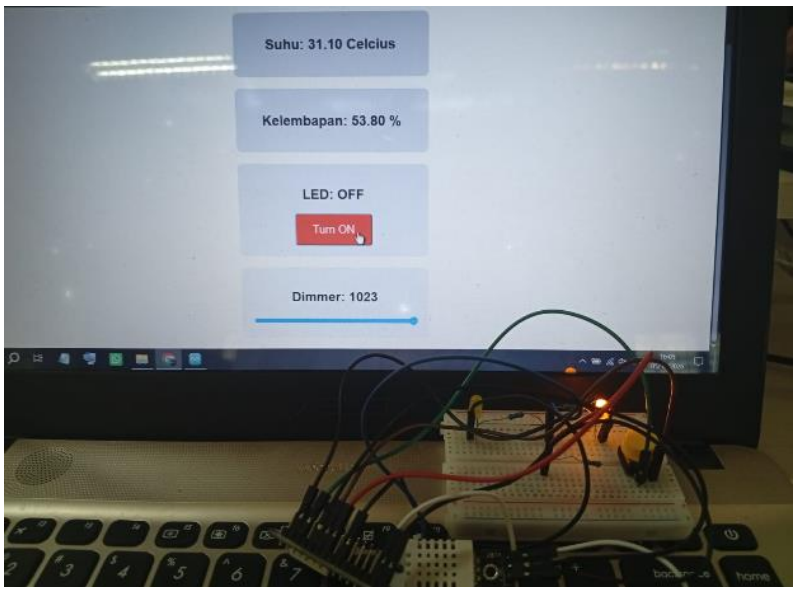
Hasilnya:

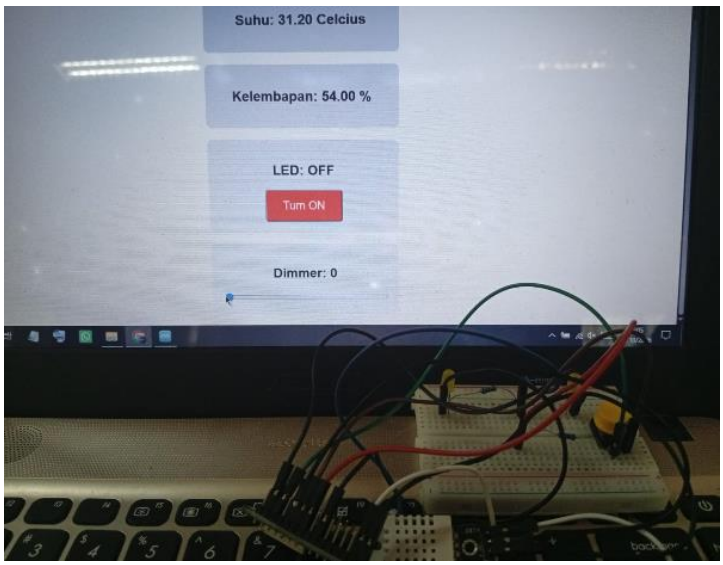


4.7 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via *WebSockets*

GAMBAR







HASIL

Sistem Kendali Intensitas Cahaya (PWM Slider) via WebSockets Sistem berhasil mengimplementasikan kontrol modulasi lebar pulsa (*PWM*) untuk mengatur intensitas cahaya secara *real-time* melalui protokol WebSockets. Berdasarkan gambar observasi antarmuka, halaman *Dashboard* web berhasil menyisipkan elemen input HTML berupa *Slider* berlabel "Dimmer" dengan rentang nilai dari 0 hingga 1023. Saat *slider* digeser hingga ke nilai maksimal (Dimmer: 1023), antarmuka mengirimkan paket *string* menuju *server* NodeMCU, lalu mikrokontroler mengeksekusi fungsi analogWrite() yang membuat LED fisik pada sirkuit menyala dengan tingkat kecerahan paling terang. Sebaliknya, saat *slider* digeser ke nilai minimum (Dimmer: 0), instruksi tersebut sukses mematikan LED secara bertahap tanpa menimbulkan jeda (lag) pada sensor lingkungan yang tetap berjalan secara paralel.

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

GAMBAR

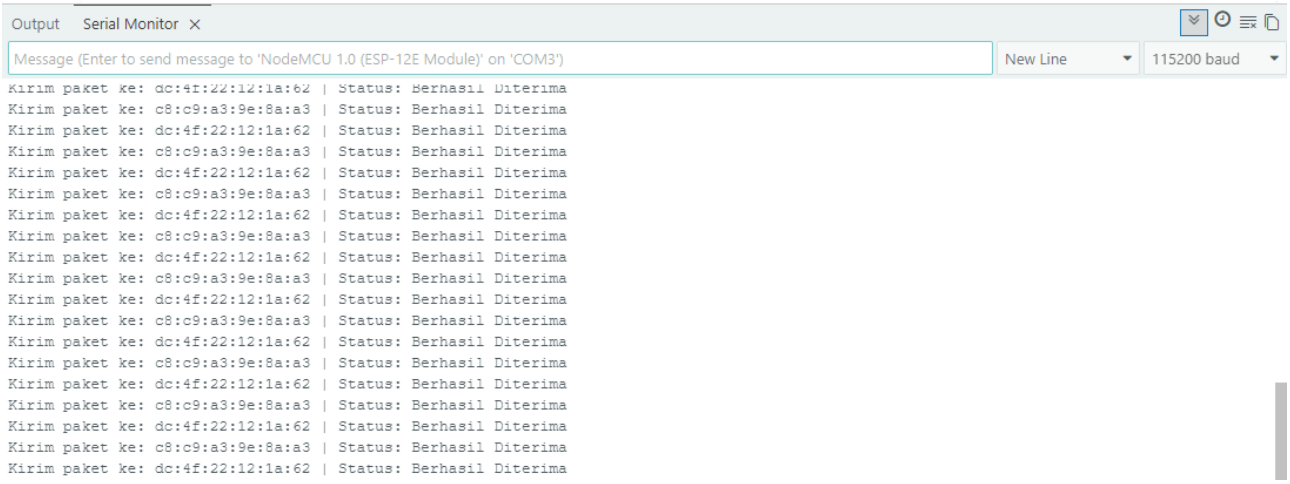


HASIL

Sistem berhasil mengaktifkan modul radio Wi-Fi internal NodeMCU ke dalam mode *Station* (STA) sehingga *firmware* mengalokasikan alamat perangkat kerasnya. Pada gambar *Serial Monitor*, mikrokontroler merespons inisialisasi tersebut dengan mencetak alamat fisik 48-bit heksadesimal pabrik, yaitu EC:FA:BC:0B:37:96. *MAC Address* inilah yang kemudian dimanfaatkan sebagai kunci identifikasi untuk membangun jalur komunikasi bebas *router* pada protokol P2P ESP-NOW.

5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

Sebagai Server/Pusatnya/Pengirim



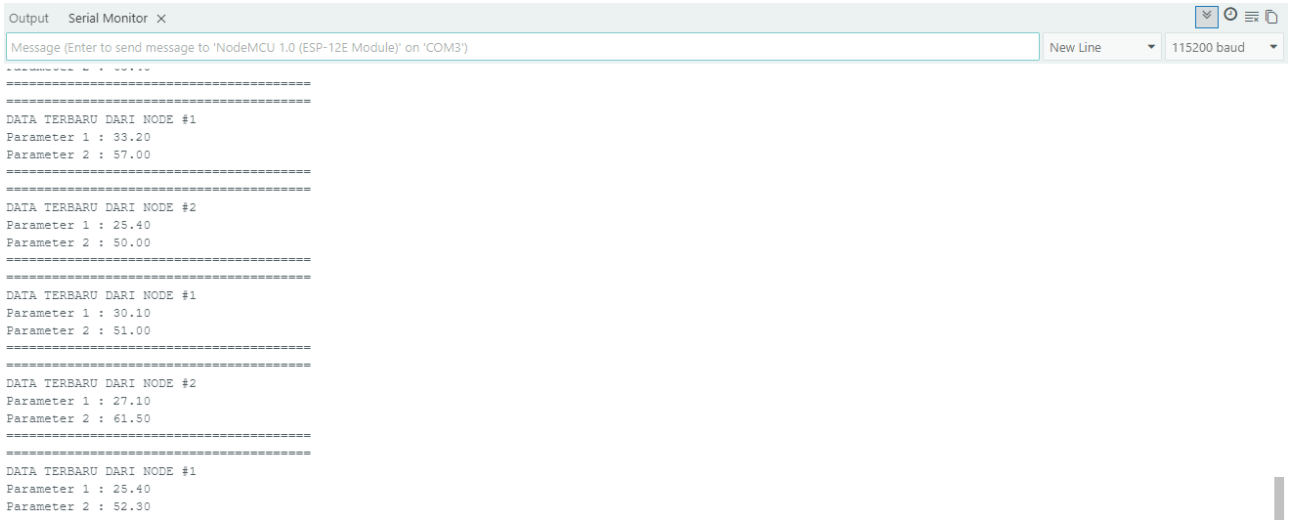
HASIL

Sistem NodeMCU (sebagai *Controller*) berhasil membangun topologi *One-to-Many* dengan menyiarkan paket data instruksi secara serentak (*broadcasting*) langsung ke dua unit *Slave* menggunakan frekuensi radio 2.4 GHz tanpa memerlukan titik akses perantara. Pada tangkapan layar *Serial Monitor*, *callback* interupsi antena sukses melaporkan status pengiriman: sistem secara bergantian menembakkan paket data ke dua alamat MAC berbeda (berakhiran 1a:62 dan 8a:a3) dan secara berurutan mencetak log "Status: Berhasil Diterima", menandakan kedua *node* berada dalam jangkauan sinyal dan merespons paket dengan baik.

Dari sisi penerima:



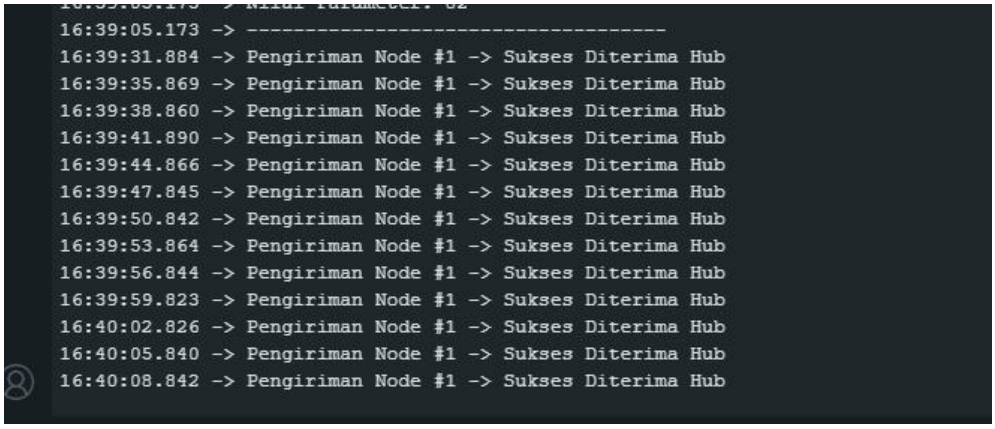
5.5.3 Praktikum 3: Pengumpulan Data Telemetri Many-to-One (ESP-NOW) Sebagai Server/Pusatnya/Pengirim



HASIL

Sistem sukses dikonfigurasikan sebagai *Aggregator Hub* (Penerima Pusat) untuk menerima struktur paket (*struct*) yang berisikan data telemetri dari beberapa sensor lapangan (*Many-to-One*). Pada log *Serial Monitor*, terlihat *Hub* mem-*parsing* memori dari gelombang radio secara bergantian dari "NODE #1" dan "NODE #2", menampilkan pemisahan parameter suhu (Parameter 1) dan parameter kelembapan (Parameter 2). Nilai parameter dari masing-masing unit tercetak dengan rapi tanpa mengalami intervensi (*memory corruption*), mengonfirmasi format struct di sisi pengirim dan penerima telah identik.

Dari sisi penerima:



5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

Sebagai Node 1



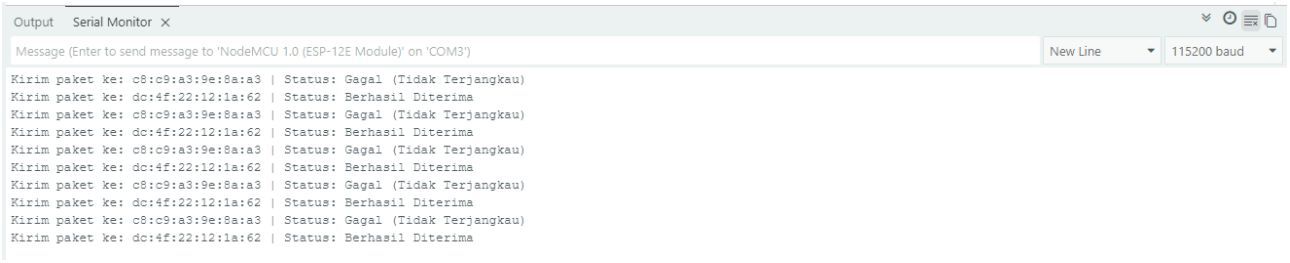
HASIL

Sistem berhasil melepaskan batasan jaringan satu arah (*single hop*) ESP-NOW dan membentuk rantai *Mesh* otonom antar-ketiga perangkat dengan memanfaatkan pustaka painlessMesh. Dari pengamatan *Serial Monitor* (pada perangkat Node-1), sistem secara berkala mencetak log [KIRIM MESH] yang memuat string format JSON berisi identitas *node*, suhu, dan kelembapan yang siap disiarkan ke semua anggota jaringan (*multi-hop broadcast*). Pada waktu yang bersamaan, *node* tersebut juga bertindak sebagai penerima, membedah struktur JSON dari *node* tetangga (Node-2 dan Node-3), lalu mencetak nilai ID jaringan unik (seperti 571611746) beserta data iklim masing-masing dengan normal.

5.6 Latihan

Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):

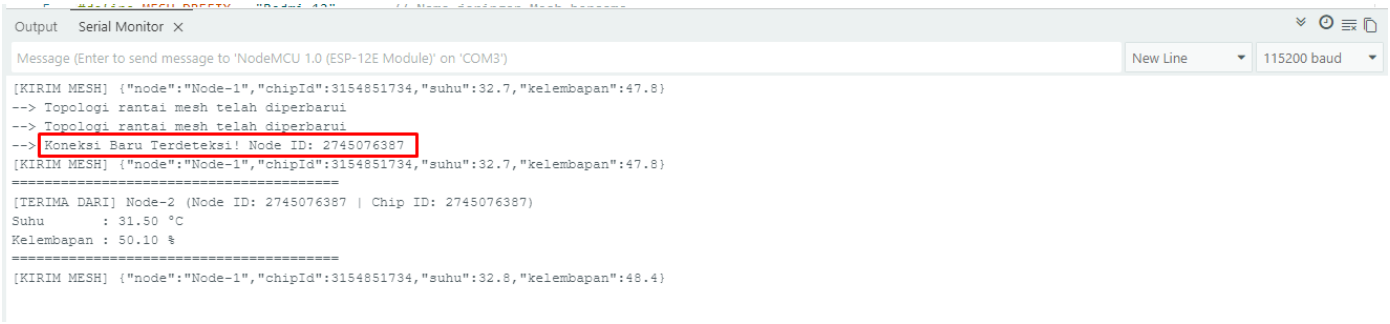
Gambar ketika salah satu penerima mencabut kabel usb dan keluarannya di controller/pusatnya



Ket: Saat kabel daya salah satu *Slave* penerima diputus secara paksa, sistem pada *Controller* berhasil mengidentifikasi kegagalan karena tidak menerima laporan (ACK). Hal ini divalidasi oleh *Serial Monitor* yang berubah secara parsial, menampilkan status "Gagal (Tidak Terjangkau)" untuk MAC Address yang putus daya (8a:a3), namun tetap mengeksekusi transmisi ke perangkat *Slave* kedua (1a:62) dengan status "Berhasil Diterima".

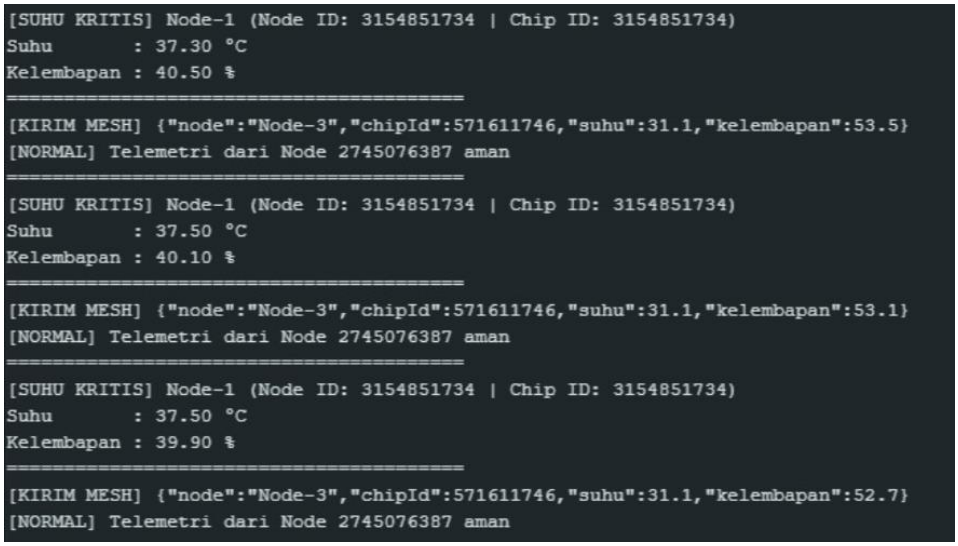
Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):

Gambar ketika salah satu penerima menekan tombol fisik RST dan tampilan ouputnya dari sisi contoller/pengirim



Ket: Ketika satu simpul *mesh* dirusak koneksinya (simulasi tombol RST), pustaka *mesh* secara reaktif mengeksekusi *callback* pemeliharaan dengan melampirkan pesan "--> Topologi rantai mesh telah diperbarui". Beberapa saat setelah *node* perantara tersebut kembali hidup, log merespons instan dengan mencetak "--> Koneksi Baru Terdeteksi! Node ID: 2745076387", membuktikan jaringan berhasil melakukan *Self-Healing* (pemulihan rute otonom) dengan sukses.

Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON - Praktikum 4):



HASIL

Sistem berhasil mengimplementasikan fitur deteksi anomali dan peringatan kondisional (*alerting*) pada aliran data di topologi jaringan *mesh*. Pada tangkapan layar *Serial Monitor*, sistem mengevaluasi ambang batas parameter iklim secara *real-time*: ketika sensor pada Node-1 mendeteksi lonjakan suhu yang tinggi (berkisar antara 37.30 °C hingga 37.50 °C), mikrokontroler seketika memicu status darurat dengan mencetak log berlabel [SUHU KRITIS] beserta rincian Node ID, Chip ID, suhu, dan kelembapannya. Di sisi lain, saat sistem menerima data telemetri dari *node* tetangga (seperti Node 2745076387) yang rentang suhunya wajar dan stabil, *firmware* memvalidasinya dengan mencetak log [NORMAL] Telemetri dari Node 2745076387 aman. Selama proses pemantauan kondisi tersebut, fungsi komunikasi *multi-hop* tetap beroperasi tanpa interupsi, dibuktikan dengan sistem yang terus menyiarkan (*broadcasting*) paket berformat JSON dari Node-3 ([KIRIM MESH]) ke seluruh anggota jaringan secara paralel.